SCORIFY

Phase 3: Model Training & Initial Results

F2602

1. Model / Algorithm Selection

Both our targets, conversion probability and predicted profit, are continuous numbers. So this is a regression problem, not classification. We picked three models to train and compare:

- **Linear Regression** — this was our baseline. It just fits a straight line through the data. We wanted to see if a simple linear model could capture the patterns. If it does well, great. If it doesn't, that tells us the data has more complex, non-linear relationships going on.
- **Random Forest** — this one builds 100 separate decision trees, each trained on a random subset of data, and then averages all their predictions together. It handles non-linear patterns well and doesn't get thrown off by outliers. We set max_depth to 10 so the trees don't get too deep and start memorizing the training data.
- **XGBoost** — this is usually the go-to model for structured data like ours. It builds trees one after another, where each new tree focuses on fixing the mistakes the previous trees made. We used a learning rate of 0.1 and max_depth of 6, which are fairly standard starting points.

2. Implementation Details

Everything was done in Python using Jupyter Notebook. These are the main libraries we used:

Library	What we used it for
pandas	Loading the CSV file and working with the data
numpy	Math operations like square root for RMSE calculation
scikit-learn	Linear Regression, Random Forest, train/test split, Label Encoding, evaluation metrics (MAE, RMSE, R^2)
xgboost	XGBoost Regressor model
matplotlib	Plotting feature importance charts and prediction scatter plots

3. Code Workflow

Here is how the pipeline works from start to finish:

- **Load cleaned data** — we start with the CSV file from Phase 1. It has 2,458 rows and 37 columns with zero missing values.
- **Drop columns based on correlation analysis** — we ran correlation analysis on all features against both targets and applied three rules to decide what to drop. Rule 1: if a feature has less than 0.05 correlation with both targets, it is not helping the model, so we drop it (destination_interest, qualified_flag). Rule 2: if two features have over 0.85 correlation with each other, they are duplicates, so we drop one (form_submissions was

identical to unknown_engagement_1, number_of_pageviews was 0.92 correlated with number_of_sessions). Rule 3: if a column is 90%+ zeros, there is barely any data in it to learn from (interaction_depth, deal_progress, nurture_score, readiness_score). After applying these three rules we dropped 8 columns.

- **Label Encoding** — we converted the remaining text columns into numbers. We went with Label Encoding instead of One-Hot because our main models are tree-based. Trees split on thresholds like "is value > 3?" so they don't care about the actual number assigned, they figure out the right splits on their own.
- **Separate features and targets** — we split the data into X (26 input features) and two targets. record_id was dropped since it is just an identifier.
- **Train/test split** — 80% for training (1,966 rows) and 20% for testing (492 rows). We used random_state=42 so the split stays the same every time we run the code.
- **Train all 3 models** — each model was trained on both targets separately.
- **Evaluate** — we compared the models using MAE, RMSE, and R^2 on the test set.
- **Feature importance and plots** — we extracted which features matter most from XGBoost and plotted actual vs predicted values.

4. Training Process

Here are the details of how we set up the training:

Parameter	Value
Train / test ratio	80% / 20% (1,966 train, 492 test)
random_state	42 (fixed so results are reproducible)
Input features	26 (14 numeric + 12 encoded categorical)
Target 1	target_conv_prob (continuous, range 0 to 0.66)
Target 2	target_profit (continuous, range 26K to 670K USD)

Hyperparameters for each model:

Model	Hyperparameters
Linear Regression	None — this model has no tunable parameters, it just fits directly
Random Forest	n_estimators=100, max_depth=10, random_state=42
XGBoost	n_estimators=100, max_depth=6, learning_rate=0.1, random_state=42

We did not do any heavy hyperparameter tuning at this stage. These are reasonable defaults that work well in most cases. The goal here was to get a working model first and see which algorithm performs best on our data.

5. Initial Results

Target 1 — Conversion Probability:

Model	MAE	RMSE	R^2
Linear Regression	0.0386	0.0652	0.4130
Random Forest	0.0238	0.0468	0.6977
XGBoost	**0.0228**	**0.0450**	**0.7202**

Target 2 — Profit Prediction:

Model	MAE	RMSE	R^2
Linear Regression	32,300	42,735	0.5851
Random Forest	**20,726**	**32,584**	**0.7588**
XGBoost	20,719	34,697	0.7265

For conversion probability, XGBoost came out on top with R^2=0.720. It explains about 72% of why some leads convert and others don't, with an average error of just 2.3%.

For profit prediction, Random Forest actually performed best with R^2=0.759, slightly ahead of XGBoost at 0.727. Both had similar MAE around 20K USD which is reasonable for deals ranging from 26K to 670K.

Linear Regression was noticeably worse on both targets, which tells us the relationships in this data are non-linear. That is why tree-based models performed significantly better.

From the feature importance analysis, behavioral engagement features came out as the strongest predictors. For conversion probability: time_on_site, total_sessions, and total_visits matter most. For profit: enquiry_touchpoints and page_interactions dominate. This lines up with what we found in Phase 1 EDA.

6. Challenges Faced

- **High cardinality columns** — destination_interest had 488 unique values and country had 132. If we one-hot encoded these, the model would end up with hundreds of extra features, most of which would just add noise. We kept country with Label Encoding since tree models can handle it, and dropped destination_interest after correlation analysis showed it had near-zero impact on both targets (0.049 and 0.007).
- **Skewed targets** — both our targets are heavily right-skewed. Most leads have a very low conversion probability (mean is only 5%) and the profit values cluster around 90K with a long tail going up to 670K. Linear models struggle with this kind of distribution. Tree-based models handle it naturally which is why they performed much better.
- **Sparse columns** — several columns were 90-99% zeros. Things like interaction_depth, deal_progress, nurture_score, and readiness_score had almost no variation. We identified these through our correlation analysis and dropped them since they were not contributing meaningful signal.
- **Duplicate features** — form_submissions and unknown_engagement_1 had a perfect 1.000 correlation, meaning they were exact copies. number_of_pageviews and number_of_sessions had 0.922 correlation. We dropped one from each pair to avoid the model double-counting the same information.

7. Why Label Encoding and Not One-Hot for Linear Regression

Someone might ask: if Label Encoding is not ideal for Linear Regression, why didn't we use One-Hot Encoding for it and Label Encoding for the tree models?

The short answer is that it would not have changed our conclusion. Linear Regression is just our baseline — we already know it will lose to the tree-based models. Even if we used One-Hot Encoding and boosted its R^2 from 0.41 to maybe 0.55 or 0.60, it would still be behind Random Forest (0.70) and XGBoost (0.72). The core problem with Linear Regression is not the encoding — it is that it can only fit straight lines, and the patterns in our data are non-linear. No encoding trick fixes that.

We chose to keep the pipeline simple and consistent: one encoding method for all three models. Since XGBoost and Random Forest were always going to be our final choices, we optimized the encoding for them. Using One-Hot would also have inflated our feature count significantly — country alone has 132 unique values which would become 131 new columns — and tree models do not benefit from that.